

Applied AI - Final Assessment Report

Dau Lara, Lecomte Anatole, Luneau Nathan

295799, 294859, 295387

January 22, 2026

1 Introduction

Among others, breast cancer is one of the most common cancers worldwide and represents a major public health concern. Indeed, according to the World Health Organization, breast cancer is the most common cancer in women in 157 countries out of 185, with an estimation of 2.3 million diagnosed and 670 000 deaths recorded in 2022.[1] Therefore, it is necessary to focus on its detection possibilities, even more when it is known that when breast cancer is detected in its earliest localized stages, the 5-year relative survival rate is close to 99%[2].

The most widely used imaging detection techniques in breast cancer diagnosis are mammography, ultrasound, magnetic resonance imaging (MRI), and positron emission tomography (PET). Although mammography is used mainly in the detection of breast cancer in asymptomatic women due to its good performance in detecting small tumors with a low dose of X-rays, the use of ultrasounds is more common for diagnosed or suspected breast cancer because it is more versatile, portable, and cost-effective than mammography, although it is highly dependent on the well-trained operator [3].

However, several issues in screening are becoming increasingly prominent in clinical practice : on the one hand large amounts of imaging data are generated thus creating a heavy workload on radiologists, and on the other hand low quality or ambiguous features are limiting the diagnostic ac-

curacy of the operator. One solution for these issues is becoming to be applied more and more : the use of artificial intelligence (AI) to assist the radiologist. Indeed, AI can be used to reduce the workload of the radiologist and increase the accuracy of its diagnostics by automated image recognition, detection and segmentation of lesions.

Related to this topic, the aim of this project is to develop and train separate AI models dedicated to medical image classification and segmentation, focusing on images presenting benign tumors (B), malignant tumors (M), or no tumors (N). In a first stage, classification and segmentation models are trained and evaluated independently in order to assess their individual performances. The main objective of the project is then to compare two different processing pipelines that combine these tasks: the first approach consists in classifying the images prior to segmenting the tumors, while the second approach starts by segmenting the tumors before performing image classification.

2 Preprocessing

To ensure high-speed data processing and eliminate bottlenecks caused by the high-latency Google Drive connection, the given dataset was replicated to a local storage using *rsync*. This allows the GPU to operate at full capacity during training.

Building on this optimized infrastructure, the

data pipeline was designed to return a combination of images, labels, and masks to support the two classification and segmentation tasks. To ensure efficient processing by the neural networks, the samples underwent geometric and statistical standardization, with preprocessing steps tailored to the requirements of each model architecture. Instead of enforcing a single fixed resolution, input images were resized according to the task and the specific model. For classification, images were resized to 128×128 pixels when training models from scratch, and to 224×224 pixels when using architectures with transfer learning. For segmentation, U-Net and DeepLabV3 operated on 224×224 inputs, whereas MaNet and SegFormer required higher-resolution inputs of 512×512 pixels to better preserve spatial details.

In addition, input normalization was not uniform across experiments but adapted to the expectations of each model. Some architectures relied on standard normalization computed from the training dataset, while others required ImageNet-based normalization to remain consistent with their backbone initialization or architectural assumptions. This model-specific normalization strategy ensured stable training dynamics while maintaining compatibility with both custom-trained and pretrained components.

To establish a robust evaluation framework, a 90/10 split was used to create training and validation sets. The splitting is stratified, ensuring that the proportions of the three classes (B, M and N) are identical in both sets, thus preventing evaluation bias and ensuring that the model is validated against all possible clinical cases.



Figure 1: Split between training and validation sets

```
=== Training ===
Classe B: 611 images (45.19%)
Classe M: 327 images (24.19%)
Classe N: 414 images (30.62%)
```

```
=== Validation ===
Classe B: 68 images (45.03%)
Classe M: 37 images (24.50%)
Classe N: 46 images (30.46%)
```

Figure 2: Split between training and validation sets

In addition to the stratified sampling, data augmentation is applied to increase the variability of the training set and improve the model’s generalization capabilities. The augmentation pipeline is implemented using the *Albumentations* library and begins with a systematic resizing of each image–mask pair to the target input resolution. A horizontal flip is then applied with a probability of 0.5. Furthermore, a geometric transformation (*ShiftScaleRotate*) introduces random translations (up to 5% of the image size), scale variations of $\pm 10\%$, and rotations of up to 15° , each activated with a probability of 0.5. All geometric transformations are applied identically to both the image and its corresponding mask to ensure perfect spatial alignment. Examples of these transformations are showed in the appendices 9.4.

3 Methods of classification and segmentation

3.1 Classification

Classification is the task of assigning an input image to one or several categories that are predefined. In the context of medical imaging for breast cancer screening, classification typically involves the determination of whether an image contains a tumor or not, and if so whether it is benign or malignant.

Modern approaches rely on machine learning and deep learning models, particularly Convolutional Neural Networks (CNN) which are able to automatically learn complex features from the raw image data. These models analyze patterns in intensity, texture, shape and spatial relationships within the image in order to distinguish between different

classes. By providing a probabilistic prediction for each category, classification models can assist clinicians in making more accurate and consistent diagnostic decisions, reduce inter-observer variability, and prioritize high-risk cases for further evaluation.

3.2 Segmentation

Segmentation refers to the process of defining the precise spatial perimeter of a structure within an image, such as a tumor, by assigning a label to each pixel (or voxel). In breast cancer imaging, segmentation is critical in order to identify tumor boundaries, to quantify the size and shape, and to monitor changes over time.

Deep learning architectures such as U-Net and its variants have proven particularly effective for segmentation tasks because they combine local and global contextual information to generate detailed masks of the tumorous bodies. Accurate segmentation provides clinicians with valuable quantitative information, supports surgical planning and enables downstream analyses such as radiomics (the extraction of quantitative features from medical images to uncover patterns and information that support diagnosis, prognosis and treatment planning) or volumetric measurements. By automating this process, segmentation models can reduce manual annotation time, enhance the reproducibility and improve the consistency of the tumor assessments across large imaging datasets.

4 Chosen models and training

The models used are based on the practical lessons 5 and 6 from the course Applied AI in Biomedicine [4] from Politecnico di Milano.

4.1 Classification

4.1.1 Build CNN from scratch

For the baseline classification, we developed a custom CNN to learn features directly from the medical images without relying on any pre-trained weights. Regarding the architecture, we implemented two different versions to find the most efficient setup. The first version used a standard flattening layer, which resulted in a very large feature vector of 32,768 elements. To simplify the model and make

it more robust against overfitting, we replaced this in the final version with Adaptive Average Pooling (Global Pooling). This reduced the feature vector to just 128 elements before it reached the dense layers, significantly cutting down the number of trainable parameters while keeping the most important spatial information.

The core of the network consists of three convolutional blocks with increasing filter depths of 32, 64, and 128. We used 3×3 kernels and Max-Pooling in each block to capture a hierarchy of features, ranging from simple edges to more complex tissue patterns. ReLU activations were applied throughout the model to introduce the non-linearity required for medical image analysis. Given the class imbalance in the dataset (with different proportions of benign, malignant, and normal cases), two training strategies were explored. A first training was performed using a class-weighted Cross-Entropy Loss to emphasize minority classes and mitigate bias toward the dominant class. A second training was then conducted using a standard Cross-Entropy Loss without class weighting. After comparison, the unweighted loss formulation was retained as it provided more stable training dynamics and better overall generalization performance on the validation set.

To stabilize the training process, we used the Adam optimizer with a learning rate of 0.001, combined with a ReduceLROnPlateau scheduler that automatically reduced the learning rate by a factor of two when the validation loss did not improve for five consecutive epochs. A dropout rate of 0.5 was applied in the fully connected layers, and Early Stopping with a patience of 10 epochs was employed to prevent overfitting and retain the best-performing model weights.

For evaluation, the validation set was used to go beyond simple accuracy. A confusion matrix 4.1.1 (Figure 3) was generated and class-wise metrics such as sensitivity and specificity were computed 4.1.1 (Figure 4), allowing a detailed assessment of the model’s ability to distinguish between benign, malignant, and normal cases. We need to use such metrics to compare between the different models 1.

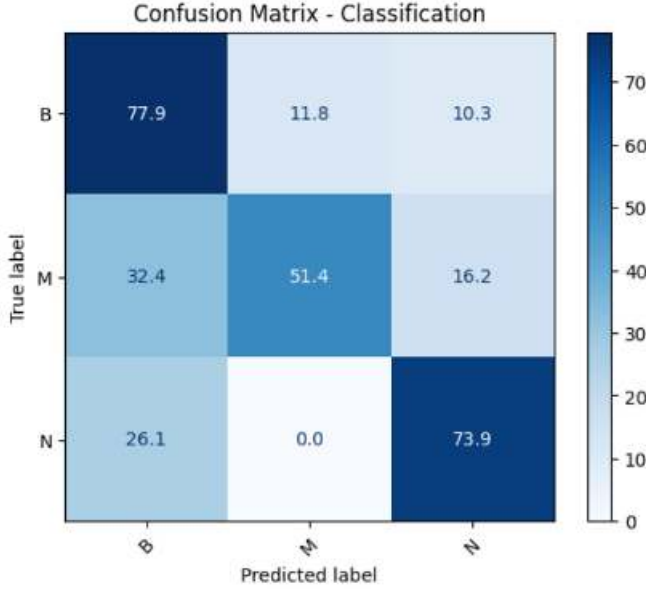


Figure 3: Confusion matrix of the model From Scratch

```

--- Global Metrics ---
Accuracy: 70.98%
Balanced Accuracy: 67.74%
F1-score (Macro): 68.51%
F1-score (weighted): 69.64%
Sensitivity (Macro): 67.74%
Sensitivity (weighted): 70.70%
Specificity (Macro): 81.06%
Specificity (weighted): 81.49%

--- Per-Class Metrics ---
B: Accuracy: 77.94% | F1: 73.10% | Sensitivity: 77.94% | Specificity: 71.86%
M: Accuracy: 51.35% | F1: 59.38% | Sensitivity: 51.35% | Specificity: 92.98%
N: Accuracy: 71.91% | F1: 71.12% | Sensitivity: 71.91% | Specificity: 80.62%

--- True vs Predicted Class Distribution ---
Class-wise distribution:
| True \ Pred | B (35.00%) | M (35.00%) | N (30.00%) |
|---|---|---|---|
| B (35.00%) | 27 (77.14%) | 4 (11.43%) | 4 (11.43%) |
| M (35.00%) | 12 (34.29%) | 26 (74.29%) | 7 (19.43%) |
| N (30.00%) | 9 (25.71%) | 0 (0.00%) | 21 (64.29%) |

```

Figure 4: Computed metrics for the from scratch CNN model

4.1.2 Transfer Learning

Transfer learning consists in reusing a model pre-trained on a large dataset as a starting point for a new task, allowing the network to leverage previously learned features; this approach is particularly effective because it reduces training time and improves performance when only limited labeled data are available.

To implement Transfer Learning, we integrated a pre-trained ResNet50 architecture into our pipeline. The core idea was to utilize a model that has already mastered feature extraction on a large-scale dataset (ImageNet), which often provides a better starting point than random initialization for medical image classification. Because ResNet50 was originally developed using ImageNet, we first

had to modify our data preparation. We created a specific dataset version using ImageNet normalization parameters ($Mean = [0.485, 0.456, 0.406]$ and $Std = [0.229, 0.224, 0.225]$). This step is important because the pre-trained convolutional filters expect input data to follow this statistical distribution. Without this step, the model would fail to recognize the trained patterns.

For the model implementation, we loaded the ResNet50 weights (IMAGENET1K_V2) and only trained the head of the layers by "freezing" the convolutional layers (setting *requires_grad = False*). This prevents the training process from updating the established weights in the backbone, essentially using the model as a fixed feature extractor. To adapt the network to our three classes (B, M, N), we replaced the default fully connected layer with a custom sequential head. This included a linear layer with 256 outputs, a ReLU activation for non-linearity, and a 0.4 Dropout layer to reduce the risk of overfitting, ending with a final linear layer for the class predictions.

The training process 9.1 was managed using the Adam optimizer and a ReduceLROnPlateau scheduler. The scheduler monitored the validation loss and halved the learning rate whenever the loss stagnated for 5 epochs, allowing for more precise weight updates in the final stages of training. To ensure the best possible results, we utilized Early Stopping with a patience of 10 epochs. This allowed us to halt the training automatically once the model stopped improving and to restore the best model for the final evaluation. This workflow ensures that we are using the most stable version of the model that generalizes well to the validation set.

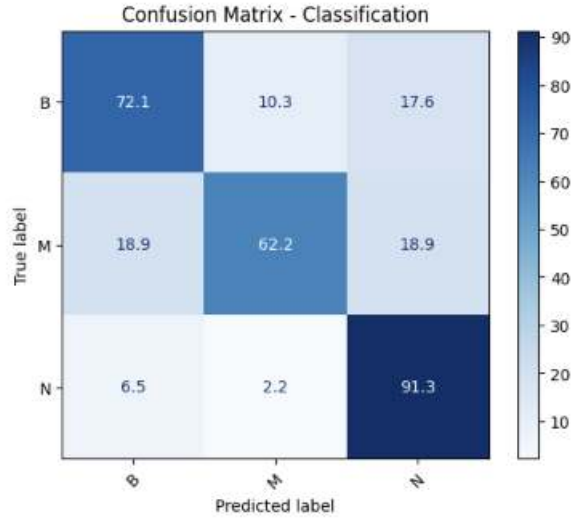


Figure 5: Confusion matrix of the model Transfer Learning

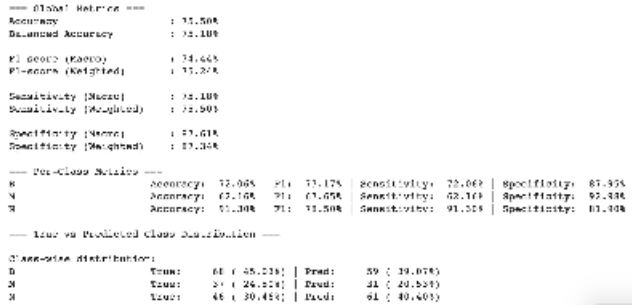


Figure 6: Computed metrics for the transfer learning model

4.2 Segmentation

In this section, we explore and evaluate four different architectures for the segmentation of medical structures: U-Net, MaNet, DeepLabV3, and SegFormer. Our goal is to identify which architecture best captures the location of the target regions. To compare these models fairly, we use two primary overlap-based metrics: Dice Coefficient and Intersection over Union (IoU).

Regarding the evaluation metrics, we implemented helper functions to calculate the Dice Score and IoU for each prediction. The Dice Coefficient is particularly important in medical imaging as it measures the overlap between the predicted mask and the ground truth, providing a value between 0 and 1. A score of 1 indicates a perfect match. Similarly, the IoU (Jaccard Index) quantifies the intersection of the two masks divided by their union.

These metrics allow us to assess the precision of the segmentation boundaries beyond simple pixel accuracy, which can often be misleading if the target object is small compared to the background.

4.2.1 U-Net from scratch

For our first segmentation experiment, we developed a Basic U-Net architecture from scratch. This allowed us to maintain full control over the model’s complexity and implement a pipeline that bypasses the normalization requirements of pre-trained models. A key advantage of this approach is the use of identity normalization ($Mean = [0, 0, 0]$, $Std = [1, 1, 1]$), which allows the network to learn directly from the raw pixel intensities of medical images, avoiding the bias of external datasets such as ImageNet.

Regarding the architectural design, the model is built around a ‘DoubleConv’ block. Each block consists of two successive 3×3 convolutions and each followed by Batch Normalization and ReLU activations. This structure stabilizes training by ensuring that the internal distributions of the feature maps remain consistent.

The network follows the classic U-shape: the contracting path uses two of these blocks and Max-Pooling to extract features at different scales, while the expansive path uses Transposed Convolutions (Up-sampling) to recover the spatial resolution. A critical technical specificity of the implementation is the use of Skip Connections (implemented via `torch.cat`). These connections combine high-resolution features from the encoder directly with the decoder. This ensures that the fine details of the tumor or tissue boundaries, which are often lost during down-sampling, are preserved for the final mask generation.

The network concludes with a 1×1 convolution and a sigmoid activation, mapping the features to a probability map between 0 and 1. Training was performed using Binary Cross-Entropy loss. During evaluation, a morphological post-processing step consisting of hole filling and opening/closing operations was applied to the predicted masks in order to reduce small artifacts and improve mask coherence. To validate the model’s effectiveness, we monitored

the Dice Coefficient throughout the training process 9.2, using Early Stopping and learning rate reductions to ensure that the model reached its peak performance on the validation set.

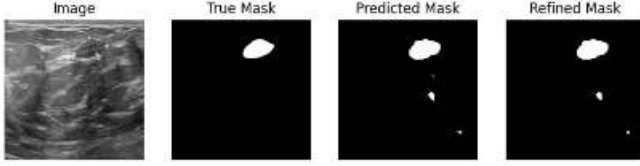


Figure 7: Predicted masks of the U-Net model

4.2.2 MaNET

Our next step was to implement MA-Net (Multi-scale Attention Network) to enable more sophisticated feature integration. This model’s key strength lies in its use of attention mechanisms to capture multi-scale context, which is often more effective than standard convolutions for varying lesion sizes. For this implementation, we used the *segmentation_models_pytorch* library with a pre-trained MiT-B0 (Mix Vision Transformer) backbone. Unlike our custom U-Net, this model required ImageNet normalization 4.1.2 to ensure the input data with the pre-trained weights of the transformer encoder.

The architecture uses a Position-wise Attention Block (PAB) and a Multi-scale Attention Block (MAB) to model global dependencies. The model was configured for binary segmentation (classes=1) and BCEWithLogitsLoss was used for better numerical stability during training. The training was managed with an Adam optimizer at a learning rate of $1e-4$, supported by a ReduceLROnPlateau scheduler and Early Stopping to prevent overfitting.

To ensure clean results, we applied the same morphological post-processing (hole filling and opening/closing) as in the previous step. This corrected minor fragmentation and produced smoother boundaries. We assessed the final performance using Mean Dice and IoU scores 2 to provide a direct comparison of the performance of the transformer-based attention backbone against the convolutional baseline.

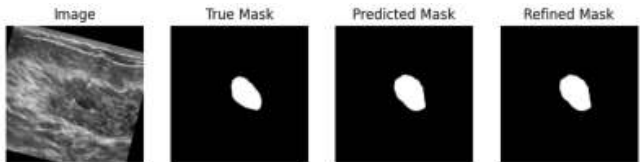


Figure 8: Predicted masks of the MaNet model

4.2.3 DeepLabV3

In the third trial, we implemented the DeepLabV3 architecture, which is known for its ability to recognize objects at various scales. We adjusted the input size to 224×224 and used a high-performance ResNet-101 backbone pre-trained on the COCO dataset. This model requires standard ImageNet normalization 4.1.2 to leverage its pre-existing weights effectively.

The defining feature of DeepLabV3 is Atrous Spatial Pyramid Pooling (ASPP). This allows the model to capture multi-scale context by using filters with different rates of ‘dilation’, which enables the model to see both fine details and the broader context without losing spatial resolution. To adapt the COCO-trained model for our task, we modified the classifier head by replacing the final 2D convolution layer to output a single channel for binary segmentation.

During training, we used BCEWithLogitsLoss and an Adam optimiser with a learning rate of $1e-4$. As DeepLabV3 is a significantly deeper network with 101 layers, we maintained a strict Early Stopping patience of 5 epochs and used the ReduceLROnPlateau scheduler to ensure stable convergence.

As with the previous models, we applied morphological post-processing (filling holes and opening/closing) to clean up the predicted masks before calculating the final Dice and IoU scores 2.

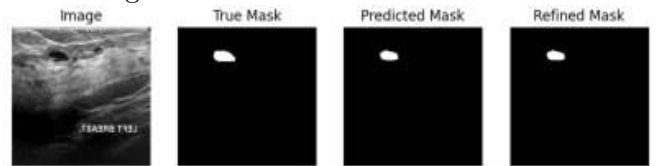


Figure 9: Predicted masks of the DeepLabV3 model

4.2.4 SegFormer

As a final segmentation experiment, we implemented SegFormer, a transformer-based architecture specifically designed for efficient and high quality semantic segmentation. Unlike convolutional encoder-decoder models, SegFormer relies on a hierarchical transformer backbone to model both local spatial patterns and long-range contextual dependencies, making it particularly well suited for

complex anatomical structures and variable lesion appearances.

For this implementation, we used the *segmentation_models_pytorch* framework with a MiT-B0 (Mix Vision Transformer) encoder, initialized with ImageNet pre-trained weights. Consequently, the same ImageNet normalization than for MaNet was applied 4.1.2 to ensure compatibility between the medical images and the transformer’s learned feature distribution. Input images were resized to 512×512 to maintain sufficient spatial resolution for precise boundary segmentation.

SegFormer departs from the classical U-shaped design by employing a lightweight, MLP-based decoder rather than explicit transposed convolutions. Multi-scale features extracted at different encoder stages are aggregated and fused to produce a dense segmentation map. This design enables the model to capture global context while preserving fine-grained spatial details, without relying on heavy skip-connection mechanisms. As a result, SegFormer achieves strong segmentation performance with lower computational overhead compared to traditional encoder-decoder pipelines.

The model was configured for binary segmentation and optimized using BCEWithLogitsLoss, ensuring stable training while learning pixel-wise probability masks. Training was conducted using the Adam optimizer with a learning rate of $1e^{-4}$, complemented by a ReduceLROnPlateau scheduler to dynamically adjust the learning rate. Early Stopping was applied to prevent overfitting and retain the best-performing model on the validation test.

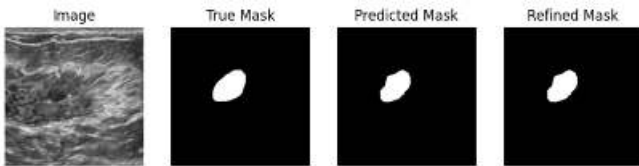


Figure 10: Predicted masks of the SegFormer model

To maintain consistency across experiments, we applied the same morphological post-processing pipeline, including hole filling and opening/closing operations. Model performance was evaluated using Mean Dice and IoU metrics 2.

4.3 Final choices

Based on the comparative analysis of our experimental results, the final choice for our pipeline is to utilize the Transfer Learning model (based on ResNet50) for the classification task and the DeepLabV3 architecture for segmentation. These models were selected because they consistently achieved the best performance across all evaluation metrics on the validation set 1 2. Specifically, the transfer learning approach significantly outperformed the CNN built from scratch by leveraging pre-trained weights, while DeepLabV3 demonstrated superior precision in delimiting tumor boundaries compared to other segmentation architectures tested in our study.

5 First approach: Classification then Segmentation

The first strategy investigated in this project follows a sequential pipeline in which image classification is performed prior to segmentation. The underlying idea of this approach is to mimic a realistic clinical workflow: a medical image is first assessed to determine whether it contains a tumor and, if so, to identify its nature (benign or malignant). Only images predicted as pathological are then forwarded to a segmentation model in order to precisely localize the tumor region.

More precisely, the pipeline is composed of two main stages. In the first stage, a three-class classification model is applied to all images of the dataset. The model used for this task is the final classification model selected in Section 4.3, namely the ResNet50-based transfer learning architecture. The model outputs one of three possible classes for each image: benign (B), malignant (M), or normal (N). This classification step acts as a decision filter and allows the system to automatically discard normal images for which no segmentation is required. The results of this 3-class classification were shown previously in the classification section 4.1.2.

In the second stage, only the images predicted as benign or malignant by the classification model are retained. The corresponding segmentation masks are then used to train and evaluate a binary segmentation model. This step strictly follows the

Model	Validation Loss	Accuracy	Balanced Accuracy	F1-score (weighted)
From scratch	0.7037	70.20%	67.74%	69.74%
Transfer learning	0.6103	75.50%	75.18%	75.24%

Table 1: Classification model comparison

Model	Validation Loss	Mean Dice score	Mean IoU score
U-Net	0.1095	0.55	0.47
MaNet	0.0729	0.77	0.71
DeepLabV3	0.0695	0.79	0.73
SegFormer	0.0759	0.73	0.66

Table 2: Segmentation model comparison

logic implemented in the code: images classified as normal by the classifier are excluded from the segmentation dataset. The segmentation task is therefore restricted to the two pathological classes (B and M), and the objective is to accurately delineate the tumor area within the image.

For segmentation, the DeepLabV3 architecture selected in Section 4.3 is used. The model is trained on the filtered dataset using binary masks, with BCEWithLogitsLoss as the optimization criterion. As in the standalone segmentation experiments, morphological post-processing operations (hole filling and opening/closing) are applied to the predicted masks in order to improve spatial coherence and reduce small artifacts. The performance of the segmentation model is then evaluated using Dice and IoU metrics 5.

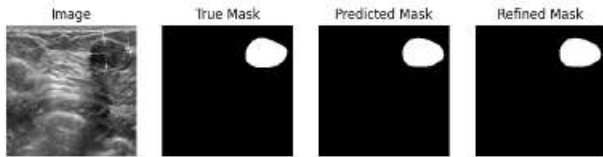


Figure 11: Predicted masks from the Task 1.b.

```

TRAINING
Evaluating: 100%|██████████| 832/832 [00:45<00:00, 18.48it/s]

Mean Dice Score: 0.89
Mean IoU Score: 0.82

VALIDATION
Evaluating: 100%|██████████| 90/90 [00:04<00:00, 19.27it/s]
Mean Dice Score: 0.80
Mean IoU Score: 0.72

```

Figure 12: Dice and IoU scores for the Task 1.b.

Overall, this approach introduces a strong dependency between the two tasks: segmentation performance is directly conditioned by the correctness

of the classification stage. Any misclassification at the first step (e.g. a pathological image predicted as normal) prevents the corresponding tumor from being segmented.

6 Second approach: Segmentation then Classification

6.1 Explanation of the approach and of the segmentation

The second strategy explored in this project reverses the order of the two tasks by performing segmentation prior to classification. Unlike the first approach, this pipeline does not rely on an initial decision about the presence or absence of a tumor. Instead, a segmentation model is applied systematically to all images in order to detect and localize potential tumor regions, which are then used as input information for the classification stage.

In this approach, the segmentation task plays the role of a region proposal mechanism. The segmentation model, chosen according to the results presented in Section 4.3, is again DeepLabV3. The model is applied to the entire dataset, including benign, malignant, and normal images. For pathological images, the model aims to extract a meaningful tumor mask, while for normal images it is expected to produce either empty masks.

Once the segmentation masks are predicted, a post-processing step identical to the one described previously (hole filling and morphological opening/closing) is applied to improve mask consistency. The results of the segmentation were previously

showed in the segmentation section 4.2.3. The resulting masks are then used to define regions of interest (ROIs) corresponding to potential tumor areas. These predicted masks are applied to the original images and form the input for the classification stage.

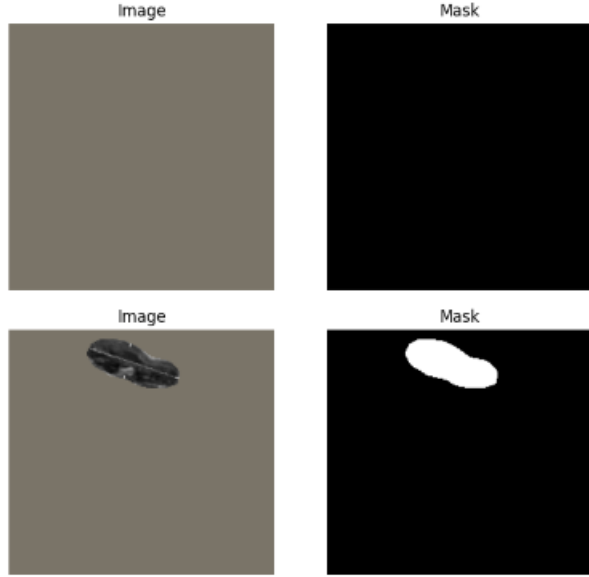


Figure 13: The predicted masks are applied to the images

6.2 Classification and choice of filter

Once the segmentation stage is completed, a classification step is required in order to distinguish between benign (B) and malignant (M) cases. However, since the segmentation model is applied to the entire dataset, including normal (N) images, an additional filtering mechanism is necessary to handle normal cases before or during classification. Two different strategies were therefore explored to perform this filtering.

6.2.1 Filter based on the segmentation

The first strategy relies exclusively on the output of the segmentation model. In principle, normal images are expected to produce empty segmentation masks, i.e., masks with no activated pixels. Assuming a near-perfect segmentation model, normal images could thus be filtered out by simply discarding all samples whose predicted masks are completely empty. This approach was implemented by removing all images associated with fully black

predicted masks. On the next figure 6.2.1, the proportion of the real labels of the discarded images is shown. We can see that the filter based on the segmentation was mostly correct, and discarded in majority images from the normal class.

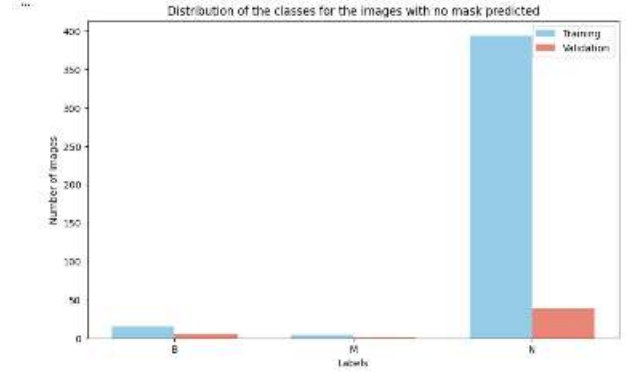


Figure 14: Filter based on the segmentation

The remaining images were then used to train a 3-class classifier (normal, benign, malignant), since in practice some normal images still receive non-empty predicted masks due to segmentation errors. The results of this 2-step classification are shown below 6.2.1. It must be said that the metrics and the confusion matrix are computed taking into account that the images filtered based on the segmentation are considered as predicted in the normal class.

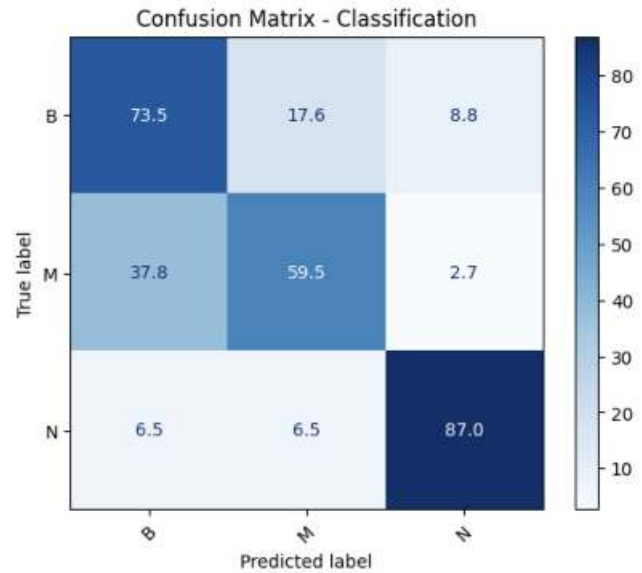


Figure 15: 3-Class Classification confusion matrix

```

--- Global Metrics ---
Accuracy : 74.174
Balanced Accuracy : 72.224
F1-score (Macro) : 72.224
F1-score (Weighted) : 74.174
Sensitivity (Macro) : 72.224
Sensitivity (Weighted) : 74.174
Specificity (Macro) : 85.524
Specificity (Weighted) : 85.524

--- Per-Class Metrics ---
0 | Accuracy: 71.224 | F1: 74.044 | Sensitivity: 73.534 | Specificity: 79.524
1 | Accuracy: 51.454 | F1: 59.444 | Sensitivity: 59.444 | Specificity: 86.044
N | Accuracy: 51.924 | F1: 86.024 | Sensitivity: 86.964 | Specificity: 93.534

```

Figure 16: 3-Class Classification metrics

This first method has the advantage of being fully independent of ground-truth class labels once the models are trained. It can therefore be applied to new, unlabeled datasets without introducing any form of data leakage or prior knowledge about the true classes. However, it does not correspond to a strict two-class classification problem, as normal images may still be present among the samples provided to the classifier.

6.2.2 Filter based on the true labels

The second strategy consists in filtering images based on their ground-truth labels. In this case, all images belonging to the normal class are explicitly removed using the true annotations, and the classifier is trained exclusively on benign and malignant samples. This results in a genuine two-class classification setting, where the model is optimized solely to discriminate between pathological cases.

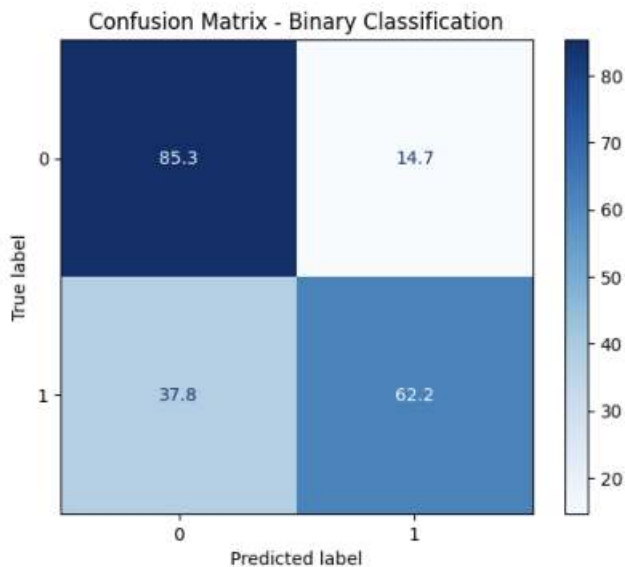


Figure 17: 2-Class Classification confusion matrix

```

--- Global Metrics ---
Accuracy : 77.164
Balanced Accuracy : 77.734
F1-score (Macro) : 74.294
F1-score (Weighted) : 75.824
Sensitivity (Macro) : 77.734
Sensitivity (Weighted) : 77.164
Specificity (Macro) : 77.734
Specificity (Weighted) : 77.164

--- Per-Class Metrics ---
0 | Accuracy: 85.234 | F1: 84.864 | Sensitivity: 85.234 | Specificity: 85.234
1 | Accuracy: 67.744 | F1: 67.714 | Sensitivity: 67.744 | Specificity: 67.714

--- True vs Predicted Class Distribution ---
0 | Class: 68 ( 64.244) | Pred: 72 ( 68.544)
1 | Class: 37 ( 35.244) | Pred: 33 ( 31.434)

```

Figure 18: 2-Class Classification metrics

While this second approach provides a clearer evaluation of benign versus malignant discrimination, it relies on the availability of true labels during the filtering step. Consequently, it cannot be directly applied to an unseen dataset without annotations and introduces a form of supervision between the segmentation and classification stages. For this reason, although it enables a cleaner two-class analysis, it is less realistic in a fully automated or deployment-oriented scenario.

7 Comparison between the two approaches

7.1 Methodologically

The two approaches represent a fundamental choice between top-down and bottom-up diagnostic logic.

- **Task 1 (Classification first) mimics the human diagnostic process:**

A doctor first determines if a patient is diseased before identifying the exact location of the problem. Methodologically, its strength lies in efficiency. By filtering out 'Normal' cases early on, the workload for the segmentation model is reduced. However, this approach creates a bottleneck: the system is only as good as the initial classifier. For example, if the ResNet50 misses a malignant tumor (False Negative), the DeepLabV3 never gets a chance to see it and the tumor remains undetected.

- **Task 2 (Segmentation first) treats segmentation as a feature detector:**

This approach assumes that if a model can identify a suspicious region, the classifier can then determine what it is. While this reduces the risk of missing a tumor due to a global classification error, if the segmentation misses

a lesion, the classifier cannot fix it. Therefore, the model must be sensitive enough to produce 'empty' masks for healthy tissue while accurately outlining even small tumors without prior knowledge of the class.

7.2 In term of results

Even though the metrics are the same, they are interpreted differently in the two tasks due to the inverted order of classification and segmentation.

Detection vs. Filtering Performance: In Task 1, the ResNet50 acts as a categorical filter. If it misclassifies a tumour as 'normal', the pipeline stops. In Task 2, specificity is mainly determined by the DeepLabV3 model. If the segmentation model produces an empty mask, the image is automatically classified as 'Normal' before it ever reaches the classifier. The role filtering between a tumor or a normal case is done by the classifier in Task 1, and by the segmentation in Task 2.

Masked Features vs. Global Context: A critical result of Task 2 is the impact of binary masking. By multiplying the input images by the predicted masks, the classifier only recognizes the lesion against a black background. While this removes background noise, it also eliminates the context (i.e. surrounding tissue) that the classifier in Task 1 utilizes. Comparing the accuracy of the two pipelines reveals whether the surrounding tissue provides necessary diagnostic clues or merely distracting noise.

Dice Score and Dataset Purity: The mean Dice and IoU scores are not directly comparable without context. Task 1 produces a 'best-case' Dice score because it is only calculated on images that have already been predicted as abnormal. In Task 2, however, the segmentation model must process the entire dataset, including 'Normal' images. Any small prediction errors on healthy tissue directly reduces the mean dice score, which is likely to lead to lower overall scores compared to the filtered evaluation in Task 1.

Analysis of the 3x3 confusion matrix: The confusion matrices highlight where each pipeline is most vulnerable. In Task 1, errors are concentrated in the initial classification stage. In Task 2, the final stage confusion matrix shows how well the model can differentiate between Benign and

Malignant using only the shape and texture of the masked lesion, independent of the rest of the image. The task 1 confusion matrix shows superior discrimination between benign and malignant tumors, but at the cost of a higher false-negative rate, with tumor cases more frequently misclassified as normal, whereas the task 2 confusion matrix exhibits the opposite trade-off and has a lower false-negative rate. In a medical context, minimizing false negatives is critical, as failing to detect a tumor poses substantially greater clinical risk than other types of misclassification, potentially leading to delayed diagnosis and treatment.

8 Discussion and conclusion

This project investigated approaches based on deep-learning for breast cancer classification and segmentation, comparing two multistage diagnostic pipelines. Overall, both classification and segmentation models achieved encouraging performances, particularly when advanced architectures were used such as ResNet50 and DeepLabV3. In addition, the use of data augmentation, post-processing and clinically relevant metrics such as Dice score, IoU, sensitivity and specificity contributed to a robust evaluation framework.

The comparative analysis revealed a key methodological trade-off. Indeed, the classification-first pipeline is computationally efficient but is highly dependent on the reliability of the initial classifier, meaning that early misclassification (e.g. false negatives in malignant cases) cannot be recovered downstream. Conversely, the segmentation-first pipeline provides greater robustness by ensuring that potential tumors are analyzed before the final classification, making it better aligned with a risk-averse clinical strategy, even though segmentation imperfections can introduce some noise in the masks.

Despite these promising results, several limitations remain, including the dataset size, some confusion between benign and malignant tumors, and error propagation between pipeline stages. Future work could focus on expanding the dataset (which has already been tried in a way with data augmentation), refining loss weighting to better handle clinically critical errors, and improving generalization across imaging conditions.

In conclusion, this project demonstrate that hybrid AI pipelines combining classification and segmentation offer a valuable foundation for computer-aided breast cancer diagnosis, with meaningful potential to assist radiologists and support early detection in clinical workflows.

References

- [1] World Health Organization, “Breast cancer,” 2025.
- [2] National Breast Cancer Foundation, Inc., “Breast cancer facts and stats,” 2025.
- [3] D. Zheng, X. He, and J. Jing, “Overview of artificial intelligence in breast cancer medical imaging,” *Journal of Clinical Medicine*, vol. 12, no. 2, p. 419, 2023.
- [4] V. Corino and M. Ferretti, “Applied ai in biomedicine: Practical lessons 5 and 6.” University course, including practical model codes, 2026. Academic year 2025–2026, Politecnico di Milano.

9 Appendices

9.1 Pictures from the training of the classification models 9.1

```
Epoch [74/100] | Train Loss: 0.7322 | Val Loss: 0.7935 | Val Acc: 66.89%
Epoch [75/100] | Train Loss: 0.7271 | Val Loss: 0.7312 | Val Acc: 71.52%
Epoch [76/100] | Train Loss: 0.7315 | Val Loss: 0.7240 | Val Acc: 70.86%
Epoch [77/100] | Train Loss: 0.7354 | Val Loss: 0.7309 | Val Acc: 70.20%
Epoch [78/100] | Train Loss: 0.7385 | Val Loss: 0.7266 | Val Acc: 70.20%
Epoch [79/100] | Train Loss: 0.7444 | Val Loss: 0.7444 | Val Acc: 70.86%
Epoch [80/100] | Train Loss: 0.7288 | Val Loss: 0.7097 | Val Acc: 72.19%
Epoch [81/100] | Train Loss: 0.7190 | Val Loss: 0.7285 | Val Acc: 69.54%
Epoch [82/100] | Train Loss: 0.7152 | Val Loss: 0.7063 | Val Acc: 71.52%
Epoch [83/100] | Train Loss: 0.7217 | Val Loss: 0.7181 | Val Acc: 69.54%
Epoch [84/100] | Train Loss: 0.7252 | Val Loss: 0.7735 | Val Acc: 67.55%
Epoch [85/100] | Train Loss: 0.7491 | Val Loss: 0.7300 | Val Acc: 69.54%
Epoch [86/100] | Train Loss: 0.7048 | Val Loss: 0.7149 | Val Acc: 69.54%
Epoch [87/100] | Train Loss: 0.7064 | Val Loss: 0.7037 | Val Acc: 70.86%
Epoch [88/100] | Train Loss: 0.7118 | Val Loss: 0.7203 | Val Acc: 69.54%
Epoch [89/100] | Train Loss: 0.7252 | Val Loss: 0.7083 | Val Acc: 70.86%
Epoch [90/100] | Train Loss: 0.7039 | Val Loss: 0.7259 | Val Acc: 70.20%
Epoch [91/100] | Train Loss: 0.7112 | Val Loss: 0.7179 | Val Acc: 70.86%
Epoch [92/100] | Train Loss: 0.7074 | Val Loss: 0.7311 | Val Acc: 68.87%
Epoch [93/100] | Train Loss: 0.7058 | Val Loss: 0.7119 | Val Acc: 72.19%
Epoch [94/100] | Train Loss: 0.6971 | Val Loss: 0.7049 | Val Acc: 71.52%
Epoch [95/100] | Train Loss: 0.6788 | Val Loss: 0.7161 | Val Acc: 70.86%
Epoch [96/100] | Train Loss: 0.7005 | Val Loss: 0.7119 | Val Acc: 70.86%
Epoch [97/100] | Train Loss: 0.6952 | Val Loss: 0.7088 | Val Acc: 70.20%
Early stopping triggered at epoch 97
Training complete. Best validation loss: 0.7037

# Save best model
torch.save(FS_model.state_dict(), r"/content/drive/MyDrive/results/cnn_from_scratch_nocw.pth")
```

Figure 1: From scratch : Loss and accuracy score during the training

```
... Epoch [1/100] | Train Loss: 0.9108 | Val Loss: 0.8209 | Val Acc: 53.64%
Epoch [2/100] | Train Loss: 0.8039 | Val Loss: 0.7530 | Val Acc: 60.26%
Epoch [3/100] | Train Loss: 0.7314 | Val Loss: 0.7929 | Val Acc: 58.28%
Epoch [4/100] | Train Loss: 0.6824 | Val Loss: 0.6544 | Val Acc: 72.85%
Epoch [5/100] | Train Loss: 0.6489 | Val Loss: 0.6881 | Val Acc: 62.91%
Epoch [6/100] | Train Loss: 0.6062 | Val Loss: 0.6472 | Val Acc: 66.23%
Epoch [7/100] | Train Loss: 0.6241 | Val Loss: 0.6859 | Val Acc: 66.23%
Epoch [8/100] | Train Loss: 0.5982 | Val Loss: 0.6434 | Val Acc: 68.21%
Epoch [9/100] | Train Loss: 0.5616 | Val Loss: 0.6447 | Val Acc: 68.87%
Epoch [10/100] | Train Loss: 0.5314 | Val Loss: 0.6455 | Val Acc: 66.23%
Epoch [11/100] | Train Loss: 0.5659 | Val Loss: 0.6387 | Val Acc: 64.90%
Epoch [12/100] | Train Loss: 0.5631 | Val Loss: 0.6203 | Val Acc: 74.83%
Epoch [13/100] | Train Loss: 0.5285 | Val Loss: 0.6143 | Val Acc: 72.19%
Epoch [14/100] | Train Loss: 0.5211 | Val Loss: 0.6461 | Val Acc: 70.86%
Epoch [15/100] | Train Loss: 0.5273 | Val Loss: 0.6537 | Val Acc: 74.83%
Epoch [16/100] | Train Loss: 0.5221 | Val Loss: 0.6103 | Val Acc: 68.87%
Epoch [17/100] | Train Loss: 0.5005 | Val Loss: 0.6706 | Val Acc: 68.87%
Epoch [18/100] | Train Loss: 0.4748 | Val Loss: 0.7072 | Val Acc: 66.89%
Epoch [19/100] | Train Loss: 0.4814 | Val Loss: 0.6371 | Val Acc: 69.54%
Epoch [20/100] | Train Loss: 0.4706 | Val Loss: 0.6576 | Val Acc: 67.55%
Epoch [21/100] | Train Loss: 0.4577 | Val Loss: 0.6466 | Val Acc: 72.85%
Epoch [22/100] | Train Loss: 0.4505 | Val Loss: 0.6590 | Val Acc: 69.54%
Epoch [23/100] | Train Loss: 0.4393 | Val Loss: 0.6236 | Val Acc: 70.86%
Epoch [24/100] | Train Loss: 0.4252 | Val Loss: 0.6293 | Val Acc: 70.86%
Epoch [25/100] | Train Loss: 0.4156 | Val Loss: 0.6226 | Val Acc: 71.52%
Epoch [26/100] | Train Loss: 0.4151 | Val Loss: 0.6141 | Val Acc: 75.50%
Early stopping triggered at epoch 26
Training complete. Best validation loss: 0.6103

# Save best model
torch.save(TL_model.state_dict(), r"/content/drive/MyDrive/results/transfer_learning.pth")
```

Figure 2: Transfer Learning : Loss and accuracy score during the training

9.2 Pictures from the training of the segmentation models 9.2

```
... cuda
Epoch [1/100]: 100%|██████████| 43/43 [00:22<00:00, 1.88it/s, dice=0.0261, loss=0.403]

Epoch [1/100] | Train Loss: 0.4739 | Train Dice: 0.1100 | Val Loss: 0.3884 | Val Dice: 0.3023
📁 Saved new best model.
Epoch [2/100]: 100%|██████████| 43/43 [00:23<00:00, 1.84it/s, dice=0.17, loss=0.347]

Epoch [2/100] | Train Loss: 0.3244 | Train Dice: 0.2515 | Val Loss: 0.2851 | Val Dice: 0.3098
📁 Saved new best model.
Epoch [3/100]: 100%|██████████| 43/43 [00:22<00:00, 1.89it/s, dice=0.262, loss=0.29]

Epoch [3/100] | Train Loss: 0.2631 | Train Dice: 0.2938 | Val Loss: 0.2514 | Val Dice: 0.2706
📁 Saved new best model.
Epoch [4/100]: 100%|██████████| 43/43 [00:21<00:00, 1.97it/s, dice=0.125, loss=0.196]

Epoch [4/100] | Train Loss: 0.2268 | Train Dice: 0.2926 | Val Loss: 0.2071 | Val Dice: 0.3086
📁 Saved new best model.
Epoch [5/100]: 100%|██████████| 43/43 [00:22<00:00, 1.90it/s, dice=0.203, loss=0.154]

Epoch [5/100] | Train Loss: 0.2040 | Train Dice: 0.2937 | Val Loss: 0.2769 | Val Dice: 0.2278
No improvement for 1 epoch(s).
Epoch [6/100]: 100%|██████████| 43/43 [00:22<00:00, 1.87it/s, dice=0.201, loss=0.224]

Epoch [6/100] | Train Loss: 0.1951 | Train Dice: 0.3088 | Val Loss: 0.1854 | Val Dice: 0.3799
📁 Saved new best model.
Epoch [7/100]: 100%|██████████| 43/43 [00:22<00:00, 1.92it/s, dice=0.48, loss=0.145]

Epoch [7/100] | Train Loss: 0.1851 | Train Dice: 0.3373 | Val Loss: 0.1893 | Val Dice: 0.3170
No improvement for 1 epoch(s).
Epoch [8/100]: 100%|██████████| 43/43 [00:21<00:00, 1.97it/s, dice=0.492, loss=0.169]

Epoch [90/100]: 100%|██████████| 43/43 [00:22<00:00, 1.89it/s, dice=0.691, loss=0.118]

Epoch [90/100] | Train Loss: 0.1140 | Train Dice: 0.5331 | Val Loss: 0.1127 | Val Dice: 0.4807
No improvement for 4 epoch(s).
Epoch [91/100]: 100%|██████████| 43/43 [00:23<00:00, 1.83it/s, dice=0.263, loss=0.104]

Epoch [91/100] | Train Loss: 0.1134 | Train Dice: 0.5368 | Val Loss: 0.1115 | Val Dice: 0.5092
No improvement for 5 epoch(s).
Epoch [92/100]: 100%|██████████| 43/43 [00:23<00:00, 1.83it/s, dice=0.585, loss=0.0811]

Epoch [92/100] | Train Loss: 0.1137 | Train Dice: 0.5138 | Val Loss: 0.1115 | Val Dice: 0.4922
No improvement for 6 epoch(s).
Epoch [93/100]: 100%|██████████| 43/43 [00:22<00:00, 1.95it/s, dice=0.726, loss=0.162]

Epoch [93/100] | Train Loss: 0.1136 | Train Dice: 0.5449 | Val Loss: 0.1099 | Val Dice: 0.5246
No improvement for 7 epoch(s).
Epoch [94/100]: 100%|██████████| 43/43 [00:22<00:00, 1.93it/s, dice=0.337, loss=0.0609]

Epoch [94/100] | Train Loss: 0.1105 | Train Dice: 0.5282 | Val Loss: 0.1109 | Val Dice: 0.5025
No improvement for 8 epoch(s).
Epoch [95/100]: 100%|██████████| 43/43 [00:23<00:00, 1.83it/s, dice=0.636, loss=0.0669]

Epoch [95/100] | Train Loss: 0.1112 | Train Dice: 0.5431 | Val Loss: 0.1105 | Val Dice: 0.4971
No improvement for 9 epoch(s).
Epoch [96/100]: 100%|██████████| 43/43 [00:23<00:00, 1.82it/s, dice=0.364, loss=0.055]

Epoch [96/100] | Train Loss: 0.1102 | Train Dice: 0.5255 | Val Loss: 0.1127 | Val Dice: 0.5028
No improvement for 10 epoch(s).
🛑 Early stopping triggered!
```

Figure 3: U-Net : Loss and dice score during the training


```

cuda
Epoch [1/100]: 100%|██████████| 43/43 [01:32<00:00, 2.16s/it, dice=0.514, loss=0.329]

Epoch [1/100] | Train Loss: 0.3877 | Train Dice: 0.3943 | Val Loss: 0.2818 | Val Dice: 0.3178
Saved new best model.
Epoch [2/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.535, loss=0.255]

Epoch [2/100] | Train Loss: 0.2865 | Train Dice: 0.5277 | Val Loss: 0.2487 | Val Dice: 0.4794
Saved new best model.
Epoch [3/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.565, loss=0.247]

Epoch [3/100] | Train Loss: 0.2357 | Train Dice: 0.5797 | Val Loss: 0.2004 | Val Dice: 0.4550
Saved new best model.
Epoch [4/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.611, loss=0.184]

Epoch [4/100] | Train Loss: 0.1986 | Train Dice: 0.6379 | Val Loss: 0.2053 | Val Dice: 0.6555
No improvement for 1 epoch(s).
Epoch [5/100]: 100%|██████████| 43/43 [01:30<00:00, 2.11s/it, dice=0.576, loss=0.224]

Epoch [5/100] | Train Loss: 0.1686 | Train Dice: 0.6622 | Val Loss: 0.1487 | Val Dice: 0.7113
Saved new best model.
Epoch [6/100]: 100%|██████████| 43/43 [01:31<00:00, 2.12s/it, dice=0.676, loss=0.137]

Epoch [6/100] | Train Loss: 0.1472 | Train Dice: 0.6941 | Val Loss: 0.1410 | Val Dice: 0.7230
Saved new best model.
Epoch [7/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.501, loss=0.208]

Epoch [7/100] | Train Loss: 0.1340 | Train Dice: 0.7004 | Val Loss: 0.1222 | Val Dice: 0.7411
Saved new best model.
Epoch [8/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.733, loss=0.112]

Epoch [8/100] | Train Loss: 0.1211 | Train Dice: 0.7178 | Val Loss: 0.1317 | Val Dice: 0.7061
No improvement for 2 epoch(s).
Epoch [15/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.821, loss=0.0989]

Epoch [15/100] | Train Loss: 0.0770 | Train Dice: 0.7661 | Val Loss: 0.0760 | Val Dice: 0.7523
Saved new best model.
Epoch [16/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.807, loss=0.0449]

Epoch [16/100] | Train Loss: 0.0728 | Train Dice: 0.7848 | Val Loss: 0.0812 | Val Dice: 0.7711
No improvement for 1 epoch(s).
Epoch [17/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.772, loss=0.093]

Epoch [17/100] | Train Loss: 0.0694 | Train Dice: 0.7871 | Val Loss: 0.0729 | Val Dice: 0.7595
Saved new best model.
Epoch [18/100]: 100%|██████████| 43/43 [01:30<00:00, 2.10s/it, dice=0.845, loss=0.0526]

Epoch [18/100] | Train Loss: 0.0653 | Train Dice: 0.8050 | Val Loss: 0.0797 | Val Dice: 0.7863
No improvement for 1 epoch(s).
Epoch [19/100]: 100%|██████████| 43/43 [01:30<00:00, 2.11s/it, dice=0.663, loss=0.0998]

Epoch [19/100] | Train Loss: 0.0640 | Train Dice: 0.7953 | Val Loss: 0.0812 | Val Dice: 0.7799
No improvement for 2 epoch(s).
Epoch [20/100]: 100%|██████████| 43/43 [01:30<00:00, 2.11s/it, dice=0.858, loss=0.0686]

Epoch [20/100] | Train Loss: 0.0611 | Train Dice: 0.7972 | Val Loss: 0.0742 | Val Dice: 0.7652
No improvement for 3 epoch(s).
Epoch [21/100]: 100%|██████████| 43/43 [01:30<00:00, 2.11s/it, dice=0.763, loss=0.0913]

Epoch [21/100] | Train Loss: 0.0631 | Train Dice: 0.7952 | Val Loss: 0.0782 | Val Dice: 0.7674
No improvement for 4 epoch(s).
Epoch [22/100]: 100%|██████████| 43/43 [01:30<00:00, 2.09s/it, dice=0.746, loss=0.0427]

Epoch [22/100] | Train Loss: 0.0582 | Train Dice: 0.8024 | Val Loss: 0.0809 | Val Dice: 0.7695
No improvement for 5 epoch(s).
Early stopping triggered!

```

Figure 4: MaNet : Loss and dice score during the training

```

... cuda
Epoch [1/100]: 100%|██████████| 43/43 [02:09<00:00, 3.00s/it, dice=0.769, loss=0.344]

Epoch [1/100] | Train Loss: 0.5001 | Train Dice: 0.5351 | Val Loss: 0.5396 | Val Dice: 0.5823
Saved new best model.
Epoch [2/100]: 100%|██████████| 43/43 [02:10<00:00, 3.04s/it, dice=0.548, loss=0.384]

Epoch [2/100] | Train Loss: 0.3184 | Train Dice: 0.7106 | Val Loss: 0.2795 | Val Dice: 0.7085
Saved new best model.
Epoch [3/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.629, loss=0.273]

Epoch [3/100] | Train Loss: 0.2549 | Train Dice: 0.7399 | Val Loss: 0.2665 | Val Dice: 0.7615
Saved new best model.
Epoch [4/100]: 100%|██████████| 43/43 [02:10<00:00, 3.02s/it, dice=0.883, loss=0.184]

Epoch [4/100] | Train Loss: 0.2097 | Train Dice: 0.7677 | Val Loss: 0.2352 | Val Dice: 0.7706
Saved new best model.
Epoch [5/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.733, loss=0.199]

Epoch [5/100] | Train Loss: 0.1785 | Train Dice: 0.8006 | Val Loss: 0.1784 | Val Dice: 0.7873
Saved new best model.
Epoch [6/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.818, loss=0.169]

Epoch [6/100] | Train Loss: 0.1568 | Train Dice: 0.7889 | Val Loss: 0.1510 | Val Dice: 0.7519
Saved new best model.
Epoch [7/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.782, loss=0.12]

Epoch [7/100] | Train Loss: 0.1353 | Train Dice: 0.8149 | Val Loss: 0.1643 | Val Dice: 0.7469
No improvement for 1 epoch(s).
Epoch [8/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.747, loss=0.117]

Epoch [8/100] | Train Loss: 0.1184 | Train Dice: 0.8358 | Val Loss: 0.1286 | Val Dice: 0.7918
Saved new best model.
Epoch [9/100]: 100%|██████████| 43/43 [02:10<00:00, 3.04s/it, dice=0.892, loss=0.0943]

Epoch [16/100] | Train Loss: 0.0555 | Train Dice: 0.8908 | Val Loss: 0.0790 | Val Dice: 0.7791
Saved new best model.
Epoch [17/100]: 100%|██████████| 43/43 [02:11<00:00, 3.05s/it, dice=0.907, loss=0.0486]

Epoch [17/100] | Train Loss: 0.0566 | Train Dice: 0.8746 | Val Loss: 0.0808 | Val Dice: 0.8012
No improvement for 1 epoch(s).
Epoch [18/100]: 100%|██████████| 43/43 [02:10<00:00, 3.04s/it, dice=0.766, loss=0.0684]

Epoch [18/100] | Train Loss: 0.0545 | Train Dice: 0.8701 | Val Loss: 0.0695 | Val Dice: 0.7742
Saved new best model.
Epoch [19/100]: 100%|██████████| 43/43 [02:11<00:00, 3.06s/it, dice=0.818, loss=0.0496]

Epoch [19/100] | Train Loss: 0.0513 | Train Dice: 0.8739 | Val Loss: 0.0873 | Val Dice: 0.7882
No improvement for 1 epoch(s).
Epoch [20/100]: 100%|██████████| 43/43 [02:10<00:00, 3.04s/it, dice=0.963, loss=0.035]

Epoch [20/100] | Train Loss: 0.0529 | Train Dice: 0.8640 | Val Loss: 0.0855 | Val Dice: 0.7908
No improvement for 2 epoch(s).
Epoch [21/100]: 100%|██████████| 43/43 [02:10<00:00, 3.04s/it, dice=0.941, loss=0.0342]

Epoch [21/100] | Train Loss: 0.0480 | Train Dice: 0.8747 | Val Loss: 0.0875 | Val Dice: 0.7927
No improvement for 3 epoch(s).
Epoch [22/100]: 100%|██████████| 43/43 [02:10<00:00, 3.02s/it, dice=0.951, loss=0.034]

Epoch [22/100] | Train Loss: 0.0420 | Train Dice: 0.8902 | Val Loss: 0.0775 | Val Dice: 0.7945
No improvement for 4 epoch(s).
Epoch [23/100]: 100%|██████████| 43/43 [02:10<00:00, 3.03s/it, dice=0.894, loss=0.045]

Epoch [23/100] | Train Loss: 0.0391 | Train Dice: 0.9036 | Val Loss: 0.0702 | Val Dice: 0.8029
No improvement for 5 epoch(s).
Early stopping triggered!

```

Figure 5: DeepLabV3 : Loss and dice score during the training

```

v  cuda
Epoch [1/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.375, loss=0.194]

Epoch [1/100] | Train Loss: 0.3291 | Train Dice: 0.2329 | Val Loss: 0.3810 | Val Dice: 0.2289
Saved new best model.
Epoch [2/100]: 100%|██████████| 43/43 [01:15<00:00, 1.76s/it, dice=0.25, loss=0.162]

Epoch [2/100] | Train Loss: 0.2154 | Train Dice: 0.3254 | Val Loss: 0.1769 | Val Dice: 0.1792
Saved new best model.
Epoch [3/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.55, loss=0.156]

Epoch [3/100] | Train Loss: 0.1949 | Train Dice: 0.3680 | Val Loss: 0.1524 | Val Dice: 0.3694
Saved new best model.
Epoch [4/100]: 100%|██████████| 43/43 [01:15<00:00, 1.75s/it, dice=0.337, loss=0.182]

Epoch [4/100] | Train Loss: 0.1813 | Train Dice: 0.3918 | Val Loss: 0.1363 | Val Dice: 0.4450
Saved new best model.
Epoch [5/100]: 100%|██████████| 43/43 [01:15<00:00, 1.75s/it, dice=0.335, loss=0.237]

Epoch [5/100] | Train Loss: 0.1603 | Train Dice: 0.4675 | Val Loss: 0.1279 | Val Dice: 0.4407
Saved new best model.
Epoch [6/100]: 100%|██████████| 43/43 [01:15<00:00, 1.75s/it, dice=0.681, loss=0.0963]

Epoch [6/100] | Train Loss: 0.1485 | Train Dice: 0.4865 | Val Loss: 0.1124 | Val Dice: 0.5274
Saved new best model.
Epoch [7/100]: 100%|██████████| 43/43 [01:15<00:00, 1.74s/it, dice=0.754, loss=0.0858]

Epoch [7/100] | Train Loss: 0.1294 | Train Dice: 0.5479 | Val Loss: 0.1064 | Val Dice: 0.4921
Saved new best model.
Epoch [8/100]: 100%|██████████| 43/43 [01:15<00:00, 1.76s/it, dice=0.564, loss=0.149]

Epoch [8/100] | Train Loss: 0.1263 | Train Dice: 0.5493 | Val Loss: 0.1024 | Val Dice: 0.5889
Saved new best model.
Epoch [9/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.511, loss=0.118]

Epoch [12/100] | Train Loss: 0.1006 | Train Dice: 0.6474 | Val Loss: 0.1129 | Val Dice: 0.6963
No improvement for 2 epoch(s).
Epoch [13/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.564, loss=0.0928]

Epoch [13/100] | Train Loss: 0.1000 | Train Dice: 0.6304 | Val Loss: 0.0987 | Val Dice: 0.7362
No improvement for 3 epoch(s).
Epoch [14/100]: 100%|██████████| 43/43 [01:15<00:00, 1.75s/it, dice=0.436, loss=0.0779]

Epoch [14/100] | Train Loss: 0.0888 | Train Dice: 0.6665 | Val Loss: 0.0806 | Val Dice: 0.6910
Saved new best model.
Epoch [15/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.761, loss=0.133]

Epoch [15/100] | Train Loss: 0.0891 | Train Dice: 0.6858 | Val Loss: 0.0759 | Val Dice: 0.7357
Saved new best model.
Epoch [16/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.501, loss=0.0656]

Epoch [16/100] | Train Loss: 0.0853 | Train Dice: 0.6759 | Val Loss: 0.1110 | Val Dice: 0.7292
No improvement for 1 epoch(s).
Epoch [17/100]: 100%|██████████| 43/43 [01:15<00:00, 1.75s/it, dice=0.572, loss=0.302]

Epoch [17/100] | Train Loss: 0.0911 | Train Dice: 0.6780 | Val Loss: 0.0972 | Val Dice: 0.7121
No improvement for 2 epoch(s).
Epoch [18/100]: 100%|██████████| 43/43 [01:14<00:00, 1.74s/it, dice=0.696, loss=0.116]

Epoch [18/100] | Train Loss: 0.0931 | Train Dice: 0.6774 | Val Loss: 0.0837 | Val Dice: 0.7108
No improvement for 3 epoch(s).
Epoch [19/100]: 100%|██████████| 43/43 [01:15<00:00, 1.76s/it, dice=0.767, loss=0.06]

Epoch [19/100] | Train Loss: 0.0805 | Train Dice: 0.7090 | Val Loss: 0.0923 | Val Dice: 0.7282
No improvement for 4 epoch(s).
Epoch [20/100]: 100%|██████████| 43/43 [01:15<00:00, 1.74s/it, dice=0.925, loss=0.0556]

Epoch [20/100] | Train Loss: 0.0752 | Train Dice: 0.7192 | Val Loss: 0.0851 | Val Dice: 0.7440
No improvement for 5 epoch(s).
Early stopping triggered!

```

Figure 6: SegFormer : Loss and dice score during the training

9.3 Pictures from the training of the tasks 9.3

```
Epoch [34/100] | Train Loss: 0.0371 | Train Dice: 0.9058 | Val Loss: 0.1059 | Val Dice: 0.8088  
No improvement for 5 epoch(s).  
Epoch [35/100]: 100%|██████████| 27/27 [01:22<00:00, 3.05s/it, dice=0.892, loss=0.0348]  
  
Epoch [35/100] | Train Loss: 0.0357 | Train Dice: 0.9086 | Val Loss: 0.1102 | Val Dice: 0.8057  
No improvement for 6 epoch(s).  
Epoch [36/100]: 100%|██████████| 27/27 [01:22<00:00, 3.06s/it, dice=0.897, loss=0.0309]  
  
Epoch [36/100] | Train Loss: 0.0346 | Train Dice: 0.9134 | Val Loss: 0.1100 | Val Dice: 0.8206  
No improvement for 7 epoch(s).  
Epoch [37/100]: 100%|██████████| 27/27 [01:22<00:00, 3.05s/it, dice=0.931, loss=0.0311]  
  
Epoch [37/100] | Train Loss: 0.0338 | Train Dice: 0.9114 | Val Loss: 0.1080 | Val Dice: 0.7849  
No improvement for 8 epoch(s).  
Epoch [38/100]: 100%|██████████| 27/27 [01:22<00:00, 3.05s/it, dice=0.921, loss=0.0407]  
  
Epoch [38/100] | Train Loss: 0.0333 | Train Dice: 0.9098 | Val Loss: 0.1037 | Val Dice: 0.7774  
No improvement for 9 epoch(s).  
Epoch [39/100]: 100%|██████████| 27/27 [01:22<00:00, 3.05s/it, dice=0.926, loss=0.0295]  
  
Epoch [39/100] | Train Loss: 0.0319 | Train Dice: 0.9112 | Val Loss: 0.1160 | Val Dice: 0.7814  
No improvement for 10 epoch(s).  
🔴 Early stopping triggered!
```

Figure 7: Task 1.b. : Segmentation : Loss and dice score during the training

Epoch [1/100]		Train Loss: 0.7523		Val Loss: 0.8577		Val Acc: 65.09%
Epoch [2/100]		Train Loss: 0.6231		Val Loss: 0.7671		Val Acc: 67.92%
Epoch [3/100]		Train Loss: 0.5677		Val Loss: 0.7521		Val Acc: 71.70%
Epoch [4/100]		Train Loss: 0.5328		Val Loss: 0.7509		Val Acc: 70.75%
Epoch [5/100]		Train Loss: 0.5059		Val Loss: 0.7622		Val Acc: 68.87%
Epoch [6/100]		Train Loss: 0.4841		Val Loss: 0.7235		Val Acc: 72.64%
Epoch [7/100]		Train Loss: 0.4960		Val Loss: 0.7278		Val Acc: 70.75%
Epoch [8/100]		Train Loss: 0.4771		Val Loss: 0.7193		Val Acc: 71.70%
Epoch [9/100]		Train Loss: 0.4545		Val Loss: 0.7589		Val Acc: 69.81%
Epoch [10/100]		Train Loss: 0.4556		Val Loss: 0.7950		Val Acc: 66.04%
Epoch [11/100]		Train Loss: 0.4243		Val Loss: 0.7870		Val Acc: 67.92%
Epoch [12/100]		Train Loss: 0.4364		Val Loss: 0.7832		Val Acc: 66.98%
Epoch [13/100]		Train Loss: 0.4250		Val Loss: 0.8183		Val Acc: 69.81%
Epoch [14/100]		Train Loss: 0.3709		Val Loss: 0.8416		Val Acc: 66.04%
Epoch [15/100]		Train Loss: 0.3919		Val Loss: 0.8742		Val Acc: 63.21%
Epoch [16/100]		Train Loss: 0.4026		Val Loss: 0.8518		Val Acc: 67.92%
Epoch [17/100]		Train Loss: 0.3615		Val Loss: 0.8334		Val Acc: 69.81%
Epoch [18/100]		Train Loss: 0.3709		Val Loss: 0.8271		Val Acc: 66.04%
Epoch [19/100]		Train Loss: 0.4074		Val Loss: 0.8242		Val Acc: 67.92%
Epoch [20/100]		Train Loss: 0.3719		Val Loss: 0.8888		Val Acc: 66.98%
Epoch [21/100]		Train Loss: 0.3293		Val Loss: 0.9025		Val Acc: 66.98%
Epoch [22/100]		Train Loss: 0.3742		Val Loss: 0.8577		Val Acc: 68.87%
Epoch [23/100]		Train Loss: 0.3416		Val Loss: 0.8505		Val Acc: 68.87%

Early stopping triggered at epoch 23

Training complete. Best validation loss: 0.7193

Figure 8: Task 2.b. : 3-class classification : Loss and accuracy score during the training

Epoch [1/100]	Train Loss: 0.5905	Val Loss: 0.5601	Val Acc: 68.57%
Epoch [2/100]	Train Loss: 0.5203	Val Loss: 0.5926	Val Acc: 70.48%
Epoch [3/100]	Train Loss: 0.4808	Val Loss: 0.5183	Val Acc: 74.29%
Epoch [4/100]	Train Loss: 0.4424	Val Loss: 0.4999	Val Acc: 75.24%
Epoch [5/100]	Train Loss: 0.4536	Val Loss: 0.4821	Val Acc: 76.19%
Epoch [6/100]	Train Loss: 0.4306	Val Loss: 0.4815	Val Acc: 76.19%
Epoch [7/100]	Train Loss: 0.3921	Val Loss: 0.5101	Val Acc: 73.33%
Epoch [8/100]	Train Loss: 0.3965	Val Loss: 0.4622	Val Acc: 79.05%
Epoch [9/100]	Train Loss: 0.3890	Val Loss: 0.5611	Val Acc: 79.05%
Epoch [10/100]	Train Loss: 0.3919	Val Loss: 0.5218	Val Acc: 74.29%
Epoch [11/100]	Train Loss: 0.3619	Val Loss: 0.4485	Val Acc: 77.14%
Epoch [12/100]	Train Loss: 0.3540	Val Loss: 0.5704	Val Acc: 72.38%
Epoch [13/100]	Train Loss: 0.3599	Val Loss: 0.4792	Val Acc: 77.14%
Epoch [14/100]	Train Loss: 0.3250	Val Loss: 0.5875	Val Acc: 73.33%
Epoch [15/100]	Train Loss: 0.3333	Val Loss: 0.5933	Val Acc: 79.05%
Epoch [16/100]	Train Loss: 0.3465	Val Loss: 0.5512	Val Acc: 78.10%
Epoch [17/100]	Train Loss: 0.3239	Val Loss: 0.5691	Val Acc: 75.24%
Epoch [18/100]	Train Loss: 0.3228	Val Loss: 0.5543	Val Acc: 75.24%
Epoch [19/100]	Train Loss: 0.3342	Val Loss: 0.5156	Val Acc: 76.19%
Epoch [20/100]	Train Loss: 0.2841	Val Loss: 0.4670	Val Acc: 73.33%
Epoch [21/100]	Train Loss: 0.2797	Val Loss: 0.4943	Val Acc: 79.05%
Epoch [22/100]	Train Loss: 0.2833	Val Loss: 0.4765	Val Acc: 77.14%
Epoch [23/100]	Train Loss: 0.2871	Val Loss: 0.7030	Val Acc: 79.05%
Epoch [24/100]	Train Loss: 0.3023	Val Loss: 0.4608	Val Acc: 75.24%
Epoch [25/100]	Train Loss: 0.3105	Val Loss: 0.6271	Val Acc: 77.14%
Epoch [26/100]	Train Loss: 0.2667	Val Loss: 0.5269	Val Acc: 77.14%

Early stopping triggered at epoch 26
Training complete. Best validation loss: 0.4485

Figure 9: Task 2.b. : 2-class classification : Loss and accuracy score during the training

9.4 Examples of the image augmentation 9.4

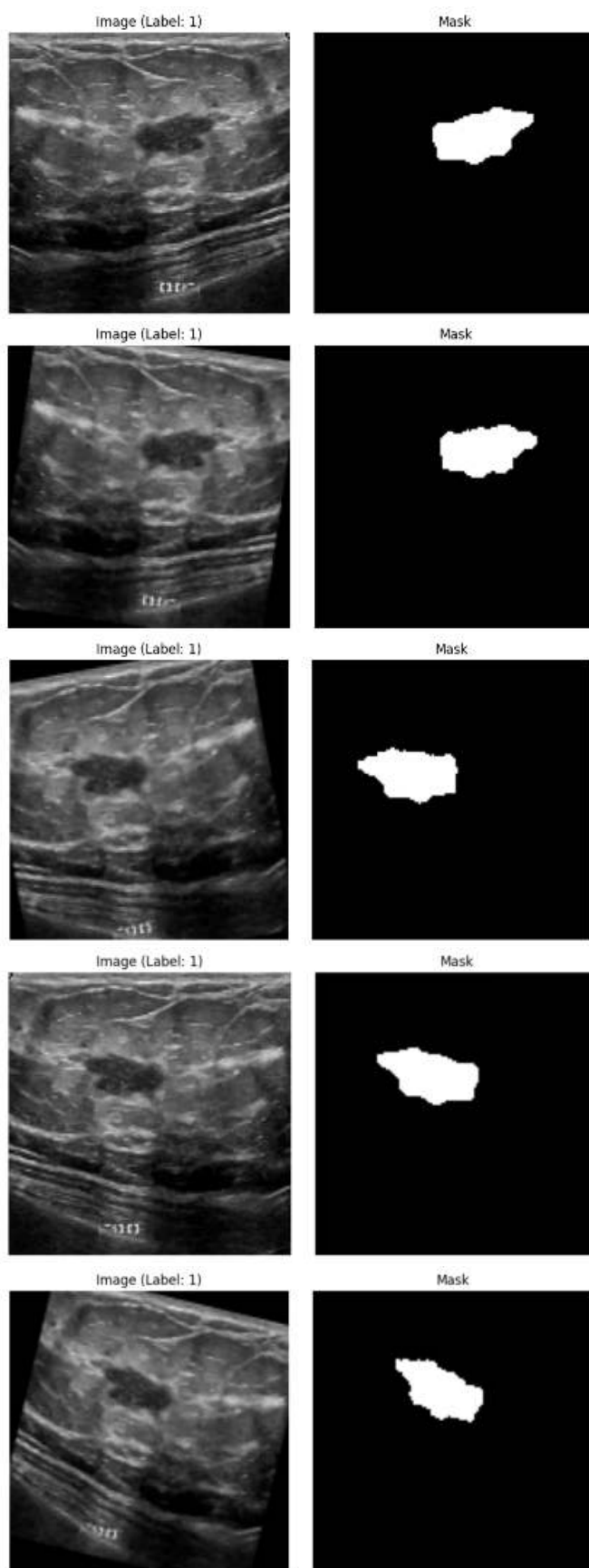


Figure 10: Transformations applied to the same image and mask